Distributed E-Commerce Order Management System

Order APIs, inventory reservation, idempotency, cache simulation, event-driven flow

Project Type	Best For	Deliverables
Backend / Distributed Systems Demo	Amazon SDE I resume + interview explanation	Runnable code, demo script, tests, PDFs

1. Problem Statement

E-commerce systems need to accept orders reliably even when users retry requests, inventory changes quickly, and payments happen asynchronously.

2. Architecture Flow

Client / Browser -> FastAPI API Gateway -> Order Service -> Inventory Reservation -> Payment Capture -> Event Log / Queue Simulation -> SQLite + Cache

3. Core Features

- Idempotent order creation using an idempotency key
- Inventory locking to avoid overselling in concurrent order flows
- Payment capture with valid state transitions
- Cache invalidation after inventory updates
- Event log representing queue-based async processing
- Automated smoke tests for order and payment workflow

4. API Surface

Endpoint	Purpose
GET /products	List products with database/cache source
POST /orders	Reserve inventory and create an order
GET /orders/{id}	Read order details
POST /payments/{id}/capture	Capture payment and emit events
GET /events	Inspect event stream

5. How to Run the Demo

Run these commands from the project folder:

```
pip install -r requirements.txt
uvicorn app.main:app --reload --port 8000
pytest -q
```

6. Demo Talking Points

I built the backend workflow that sits behind an e-commerce checkout. The important part is not the UI; it is handling duplicate requests, inventory consistency, payment state, and event-driven processing in a predictable way.

7. Resume Bullet

> Built a scalable order-management backend with idempotent order APIs, inventory locking, payment state transitions, cache invalidation, event logs, Docker support, and automated tests.

8. What to Say in Interview

- Why I built it: to practice backend workflows that need correctness, reliability and scalability.
- Main challenge: keeping state transitions correct when retries or failures happen.
- What I learned: data structures, APIs, testing, failure handling, and clean modular design.
- Future scale-up: replace in-memory components with Redis/Kafka/SQS, add auth, metrics, tracing and cloud deployment.

9. Demo Output Snapshot

GET /products


```
{
"source": "database",
"products": [
{
"id": 1,
"name": "Mechanical Keyboard",
"price": 4500,
"stock": 8
},
{
"id": 2,
"name": "Noise Cancelling Headphones",
"price": 6500,
"stock": 5
},
{
"id": 3,
"name": "USB-C Hub",
"price": 2500,
"stock": 12
}
]
}
```


POST /orders

```
{
"deduplicated": false,
"order_id": 1,
"status": "RESERVED",
"amount": 4500
}
```


POST /payments/{id}/capture


```
{
"order_id": 1,
"status": "PAID",
"amount": 4500
}
```